

成都猫狗小院

流浪猫狗领养平台 · 技术文档

「传智杯」全国 IT 技能大赛 · AI WEB 网页开发挑战赛

赛项主题：健康守护

线上地址：chuanzhibei-d3gvmowp1e63d7f33-1470251683.tcloudbaseapp.com

2026 年 10 月

目录

图表清单.....3
一、项目概述.....4
二、需求分析.....5
三、系统架构设计.....7
四、AI 技术方案.....11
五、核心功能实现.....19
六、测试与部署.....24
七、创新点.....28
八、总结与展望.....29

提示：如 Word 未自动刷新，请选中目录后按 F9。

图表清单

图表清单

编号	名称
图 1	整体系统架构
图 2	数据模型与权限边界
图 3	数据访问的两条通道
图 4	向量模型语义方向实测
图 5	Agent 工具调用时序
图 6	RAG 检索与按需重建索引
图 7	AI 请求链路与降级路径
图 8	访客侧 AI 匹配面板
图 9	管理员侧 AI 审核助手
图 10	以图搜宠结果
图 11	语音输入与朗读
表 1	名词约定
表 2	非功能性需求
表 3	云函数职责划分
表 4	模型服务分工
表 5	六项 AI 能力与解决的问题
表 6	文本向量模型对比
表 7	六个工具
表 8	SSE 事件类型
表 9	三层降级行为
表 10	提交侧的约束措施
表 11	测试覆盖
表 12	测试拦住的缺陷

一、项目概述

1.1 项目背景

成都猫狗小院是真实在运营的私人流浪动物救助站。本项目的服务对象不是虚构场景，小院每天在处理真实的领养咨询与申请。

平台的**基础版本**于 2026 年 9 月上线并投入使用，包含宠物档案展示、领养申请提交、管理员后台、企业微信新申请提醒四项功能，由项目负责人独立开发完成。

本次参赛是在这个已经运行的平台之上做增量，不是从零搭建。

1.2 参赛期间的工作范围

参赛期间（2026 年 10 月起）的工作集中在两处：

一是 AI 能力。 把平台从「能收申请」推进到「能判断什么样的领养更可能成功」。共实现六项 AI 能力，覆盖访客侧与管理员侧。

二是工程质量。 引入 ESLint 与 Prettier、补环境变量模板和验证脚本、修复几处历史问题（包括一处导致后台多个功能长期不可用的缺陷）。

全部改动记录在仓库提交历史中；本轮多页与测试整改也按职责分批提交。

1.3 与赛项主题的关系

赛项主题是「健康守护」。流浪动物救助语境下的「健康」，不只指动物本人的身体状况，更指向**领养关系本身的健康度**。

一次不匹配的领养，结果通常是二次遗弃。动物经历一次被领养再被退回，伤害往往大于从未被领养。而绝大多数不匹配，在申请阶段通常可以识别——租房没有经房东同意、家庭成员意见不一致、白天无人陪伴、居住地不适合养大型犬。

本项目做的是把这些问题**提前到申请之前和审核之中**发现。

1.4 名词约定

表 1　名词约定

名词	含义
宠物档案	宠物的种类、性别、年龄、性格描述、健康状况、标签
领养要求	这只宠物对应的具体要求，如封窗、家庭成员同意

名词	含义
回访记录	领养后的跟踪记录，含日期、标题、正文
工具（Tool）	模型可以主动调用的服务端函数，本项目共六个
向量检索	把文本或图片转成向量，按语义相似度而非关键词匹配
特权通道	云函数内部以表属主身份访问数据库的通道，绕过行级安全策略

二、需求分析

2.1 救助站的实际处境

小院目前只有**一位管理员**，负责全部宠物档案维护、申请审核、领养后回访。申请在真实进来，每一条都要人工读完，再对照这只宠物的领养要求逐项核对。

这个处境决定了两条设计约束：

约束一：AI 不能增加工作量。 任何需要管理员额外维护数据、额外点击确认的设计都要排除。

约束二：AI 不能替代判断。 管理员的时间紧，但录取决定不能交给模型，出错的社会成本由动物承担。

2.2 访客侧需求

需求 1：不知道该选哪只。 访客通常凭照片产生好感，但照片看不出是否适合自己。现有的按种类、性别、状态筛选，只能表达"我要猫"，表达不了"我要一只不用天天遛的猫"。

需求 2：描述性需求表达不出来。 这是关键词筛选的根本局限。当访客说「想找一只安静点的」，而档案里写的是「不爱动，大部分时间在睡觉」时，关键词匹配返回空结果，但语义上它们是同一件事。

需求 3：部分访客不擅长打字。 小院的申请人群中包含年长申请人。要求他们用文字描述居住情况、作息、家人态度，本身就是一道门槛。

2.3 管理员侧需求

需求 4：逐条读申请耗时。 一份正式申请包含姓名、年龄、性别、联系方式、住房情况、养宠经验、家庭成员态度、现有宠物、补充说明。管理员需要从中提取要点，再对照要求找缺口。

需求 5：需要知道申请人是否重复投递。 同一人在短时间内向多只宠物提交申请，是一个值得注意的信号，但人工核对需要翻遍历史记录。

需求 6：需要知道宠物此前的回访情况。 判断一只宠物是否适合再次开放领养，历史上的领养结果很重要。

2.4 非功能性需求

表 2　非功能性需求

需求	说明
隐私保护	申请人提交的是个人信息，不得进入公开仓库、不得发给外部模型
成本可控	赛事要求使用商用 API 须自行承担费用，需控制在免费额度内
服务可用	AI 服务异常不得影响领养申请提交这条核心业务
可维护	小院没有专职开发者，代码需要负责人自己能看懂、能长期维护
如实表达	AI 不得编造档案里没有的信息，不得对未公布的政策作推测

2.5 明确不做的事

需求分析同样包括划定边界。以下四件事**明确不做**：

不做录取决定。 AI 只做信息整理与提示，最终决定权在管理员。项目在提示词、界面文案、系统提示三处都写明了这一点。

不做内容生成。 不生成宠物照片、不生成宣传文案。救助站的素材应当来自真实记录。

不做社交功能。 不引入评论、点赞、关注。平台的目的是促成领养，不是留住用户。

不做视频分析。 多模态只覆盖文本、图片、语音三种模态。视频与领养场景关联弱，加入后并无实际用途。

三、系统架构设计

3.1 整体架构

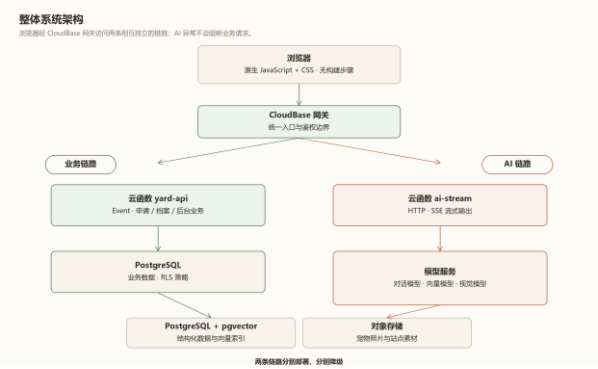


图 1　整体系统架构。平台由静态前端、两个云函数、PostgreSQL 数据库与对象存储组成。两条请求链路相互独立，AI 服务异常不影响业务请求

系统分为四层：

前端层。 多页静态站点，原生 JavaScript 与 CSS，无构建步骤。七页为 `index.html`、`pets.html`、`match.html`、`adoption.html`、`stories.html`、`about.html`、`admin.html`。页面使用普通链接跳转，共享 `app.js`、`styles.css` 和 `site-config.js`；线上通过 CloudBase 网关访问后端。

业务后端。 云函数 `yard-api`，Event 类型。负责申请提交、修改令牌校验、后台管理接口、企业微信提醒。

AI 后端。 云函数 `ai-stream`，HTTP 类型。负责 SSE 流式对话、工具调用、向量检索、图像比对。

数据层。 PostgreSQL（CloudBase RDB）承载全部业务数据与向量数据，对象存储承载宠物影像。

3.2 云函数拆分

表 3　云函数职责划分

云函数	类型	职责	超时
`yard-api`	Event	申请提交与修改、后台管理、企业微信提醒、管理员侧 AI 审核	15 秒
`ai-stream`	HTTP	SSE 流式对话、工具调用、RAG 检索、以图搜宠	60 秒

为什么 AI 必须单独一个函数。 这不是架构偏好，是平台限制：SSE 流式响应只在 HTTP（Web）类型的云函数上支持。`yard-api` 是 Event 类型，经网关调用只返回一次性 JSON，无法边生成边推送。

拆分的附带收益是风险隔离。AI 服务涉及外部模型接口、向量服务、图像抓取，不确定性高于业务逻辑。放在独立函数里，AI 出问题不影响申请提交这条主链路。

3.3 技术选型

3.3.1 前端：多页静态站点

理由。 站点是纯静态的，无构建步骤，复杂度和首屏体积都最低。更重要的原因是可维护性：小院没有专职开发者，代码需要负责人自己能看懂。引入构建工具链后，任何一次依赖升级、构建配置变更都可能让项目卡住，而没有人能处理。

为什么拆页。 原单页将浏览宠物、AI 咨询、领养流程、回访与小院介绍放在同一长页面。分享只能指向同一文档的锚点，无法单独呈现匹配任务；访客需要在多个任务区之间反复定位，屏幕阅读器也需跨过较长的无关内容。改为每个任务一个真实 URL，如 `match.html`，可直接打开、分享或刷新。仍使用普通链接和一份共享样式、脚本，没有引入前端框架或构建步骤。

代价。 评分标准中「TypeScript 规范使用」「组件化架构设计」两项无法得分。我们接受这个代价，把工程复杂度的预算集中投入到占权重 35% 的 AI 部分。这个取舍在第七章创新点中会进一步说明。

3.3.2 数据库：PostgreSQL + 行级安全策略

理由。 权限不能依赖前端自觉。CloudBase RDB 支持 PostgreSQL 的行级安全策略（RLS），可以把「谁能读哪一行」写在数据库里，前端即使被篡改也读不到不该读的数据。

项目实施中所有表均启用 RLS。例如 `yard_administrators` 表只对 `authenticated` 角色开放，且必须满足 `is_yard_admin()` 判定；匿名用户可以读宠物档案，但没有对任何业务表的写权限。

3.3.3 向量检索：pgvector

理由。 数据库原生支持向量类型与近似最近邻索引，不需要额外引入向量数据库。数据规模在数百到数千条量级，单库即可承载，运维成本最低。

维度选择。 pgvector 的近似最近邻索引（HNSW、IVFFlat）对 `vector` 类型只支持 2000 维以内。这一限制直接决定了两处选型：

- 文本向量选择 `kinfra-text-embedding-0.6b`（1024 维），可以建立 HNSW 索引。

质量更高的 4b 模型输出 2560 维，超出索引上限，规模增大后只能全表精确扫描，反而更慢。

- 图片向量模型 `kinfra-vl-embedding-2b` 固定输出 2048 维，同样超出 `vector` 的索引上限。这里改用 `halfvec`（半精度，索引上限 4000 维）承接，代价是极小的精度损失，换来可以走索引检索。

3.3.4 模型服务：两个供应商分工

表 4　模型服务分工

用途	服务	模型
对话与工具调用	DeepSeek	`deepseek-chat`
文本向量	腾讯云 TokenHub	`kinfra-text-embedding-0.6b`
图片向量	腾讯云 TokenHub	`kinfra-vl-embedding-2b`
图片理解	腾讯云 TokenHub	`hy-vision-2.0-instruct`

为什么对话与向量分开。 DeepSeek 提供对话能力，但不提供向量化接口。向量化需要另外的服务。选择腾讯云 TokenHub 的原因是它与现有云资源同源，凭证与网络链路最短；同时它一个模型同时覆盖文本向量与图片向量，不需要为多模态再引入第二个供应商。

3.4 数据模型



图 2　数据模型与权限边界。业务表与向量表的权限边界。向量表不对外开放，仅供云函数经特权通道访问

项目共 10 张表，按用途分为三类：

业务表。 `pets`（宠物档案）、`pet_media`（影像）、`pet_updates`（回访记录）、`applications`（领养申请）、`application_events`（状态变更记录）、`yard_administrators`（管理员）、`yard_settings`（站点设置）。

限频表。 `rate_limit_events`，记录限频事件。IP 只以 SHA-256 哈希形式存储。

向量表。 `knowledge_chunks`（文本向量，1024 维）、`pet_photo_vectors`（图片向量，2048 维）。

向量表启用 RLS 且不建立任何策略，同时回收了 `anon` 与 `authenticated` 的授权。这意味着客户端无法直接读取。云函数经特权通道访问，不受 RLS 限制。

3.5 数据访问的两条通道



图 3　数据访问的两条通道。前端直连数据库受 RLS 约束；云函数经特权通道访问，可读写受限表

> 这是本项目最重要的一处工程决策，也是**一次真实缺陷的产物**。

通道一：前端直连。 前端通过 CloudBase SDK 直接访问数据库。读到的数据由 RLS 策略决定，写操作同样受策略约束。用于读取公开宠物档案、公开站点设置，以及管理员读取申请列表。

通道二：云函数特权通道。 云函数通过腾讯云 OpenAPI 的 `ExecutePGSql` 接口执行 SQL，以表属主身份运行，绕过 RLS。用于所有需要写入或读取受限数据的场景。

为什么必须分开。 云函数内部的 `@cloudbase/node-sdk` 提供的 rdb() 客户端以**匿名身份**运行，而匿名角色在这些表上没有授权。项目在开发过程中把 `requireAdmin` 与管理类操作写在了 rdb() 上，导致后台的申请审核、宠物档案保存、收件箱读取等功能**长期不可用**，报错为 `permission denied for table`。

这个问题一直没有暴露，直到新增删除功能时才被撞出来——因为它只在真正调用到那些功能时才会出现，而此前从未被测试覆盖。

修复方式是把这些操作全部迁移到特权通道。当前项目的规则是：**云函数内需要访问受限数据时必须使用特权通道，不得使用 rdb()。**

四、AI 技术方案

表 5　六项 AI 能力与解决的问题

能力	面向	解决的问题
领养匹配助手	访客	不知道该选哪只
申请审核助手	管理员	逐条读申请耗时
语义检索（RAG）	访客	描述性需求表达不出来
语音输入与朗读	访客	不擅长打字
照片理解	管理员	撰写档案描述耗时
以图搜宠	访客	说不清想要什么，但能指一张照片

4.1 总体设计

AI 部分不是"在页面上加一个对话框"，而是四层能力叠起来：

```
多模态层    语音输入与朗读 · 照片理解 · 以图搜宠
Agent 层    访客侧匹配助手 · 管理员侧审核助手
知识层      RAG 语义检索（pgvector + 向量模型）
工具层      六个只读工具（模型自主调用）
```

上层依赖下层。工具层让模型能查真实数据，知识层让查询按语义而不是关键词，Agent 层让模型自己决定查什么、查几次，多模态层把输入从文字扩展到语音和图片。

4.2 选型理由

4.2.1 为什么不把宠物档案直接塞进提示词

这是整个方案里最先要回答的问题。

最初版本的实现，是把全部已发布宠物的档案拼接成一段文本放进系统提示词。6 只宠物时约占 800 个 token，尚可运行。

问题有两个：

一是容量不足。 真实救助站在册动物几十到几百只。按每只 130 token 估算，100 只是 1.3 万 token。不仅成本高，而且会挤压对话空间、拉低模型对关键信息的注意力。

二是无法回答。 领养政策、回访记录这两类知识根本没进提示词。访客问「你们怎么回访」「领养要收费吗」，模型只能编。这不是提示词写得不够好，是信息压根不在上下文里。

因此改为工具调用：模型拿到的是「有哪些工具」，需要什么自己去查。档案涨到几百只也不影响；回答基于实时数据；每一次查询还能留痕，在界面上显示成「正在按语义检索小院资料…」。

4.2.2 对话模型：DeepSeek

选择依据有三条：

支持 Function Calling。 这是硬条件，不支持工具调用就退回成了纯问答。

接口兼容 OpenAI 格式。 请求与响应结构不需要额外适配层，工具调用、流式响应、JSON 输出都是标准写法，后续换模型改动很小。

价格可承受。 按量计费，本项目在免费额度内即可完成开发与演示。

4.2.3 向量模型：TokenHub，且刻意选小不选大

DeepSeek 提供对话能力，**但不提供向量化接口。**向量化必须另找服务。

选择腾讯云 TokenHub 的原因是它与现有云资源同源：凭证复用、网络链路最短，不需要为多模态再引入第二个供应商——它的多模态模型同时支持文本和图片输入。

TokenHub 提供两个文本向量模型：

表 6　文本向量模型对比

模型	输出维度	官方定位
`kinfra-text-embedding-0.6b`	1024	大规模召回、延迟与成本敏感场景
`kinfra-text-embedding-4b`	2560	高质量文本检索、深层语义理解

我们选了维度更低、定位更"廉价"的那个。 理由不是省钱，是索引限制：

pgvector 的近似最近邻索引对 `vector` 类型**只支持 2000 维以内。**2560 维建不了 HNSW 索引，只能全表精确扫描——数据量一大反而更慢。1024 维能建索引，在本项目的语料规模与语义复杂度下足以满足需要。

这是一个"看起来更差、实际更好"的选择，也是本次方案中少数几个先看约束、再看参数的决定。

选定后做过一次语义方向验证，确认接口能用不等于检索方向正确：



图 4　向量模型语义方向实测。取三句固定文本（两句属于领养场景、一句与领养无关），分别向量化后比较余弦相似度。相关句之间的相似度明显高于与无关句的相似度，说明检索方向正确。模型 kinfra-text-embedding-0.6b，输出 1024 维，单次调用约 350ms

4.2.4 向量存储：pgvector + halfvec

为什么不引入专用向量数据库。 本项目的向量规模在数百到数千条量级（当前文本向量 14 条、图片向量在建）。独立向量库带来的是额外的部署、额外的凭证、额外的一致性维护，而收益在这个量级上接近于零。CloudBase RDB 原生支持 pgvector，一个库解决。

图片向量的维度问题。 多模态模型 `kinfra-vl-embedding-2b` 固定输出 2048 维，又超出了 `vector` 的 2000 维索引上限。这次没有降级换模型，而是改用 `halfvec`（半精度浮点，索引上限 4000 维）。代价是极小的精度损失，换来能走索引检索。

4.2.5 为什么用 Function Calling 而不是 MCP

评分标准把这两者并列（「是否通过 MCP 协议**或** Function Calling 实现集成」）。我们选 Function Calling，原因是场景：

MCP 解决的是「让外部 AI 客户端接入你的能力」，适合把服务暴露给 Claude Desktop 这类通用客户端。而本项目的模型调用发生在自己的云函数内部，工具就是同进程的函数，引入 MCP 只是在中间加一层协议转换，不带来任何能力增益，却增加了部署与调试成本。

工具层的接口设计与 MCP 是同一个思路（名称、描述、JSON Schema 参数），将来若要对外暴露，改造成 MCP Server 的成本很低。这一点在第八章展望中会提到。

4.2.6 为什么不做视频模态

多模态评分项涵盖文本、图像、语音、视频。我们只做了前三种。

视频与领养场景的关联很弱：访客不会因为一段视频决定领养哪只动物，管理员也不需要从视频里提取信息。为了凑齐模态而加入视频分析，换来的是一个没人用的功能和一个更难维护的系统。

需求分析里明确写下的"不做的事"，在这里兑现了。

4.3 集成方式

4.3.1 工具层

表 7　六个工具

工具	作用	关键设计
`search_knowledge`	按语义检索全部文字资料	描述性需求走这条
`search_pets`	按种类、性别、关键词检索	条件明确时走这条，更快更准
`get_pet_profile`	读取某只宠物完整档案	推荐前确认细节
`get_adoption_policy`	读取领养政策原文	未填写时明确要求模型不许推测
`get_follow_up_history`	读取某只宠物的回访记录	判断是否适合再次开放领养
`check_requirement_gaps`	逐条对照常见领养要求	规则负责找缺口，模型负责讲道理

`search_knowledge` 与 `search_pets` 并存是有意的：描述性、说不清条件的需求走语义检索，条件明确的走结构化查询。两者都提供给模型，由它自己判断选哪个。

`check_requirement_gaps` 需要单独说明。它用关键词匹配判断访客"有没有提到"某件事（封窗、家人意见、白天是否有人等），返回的是「已说明 / 未提及」，**而不是「满足 / 缺失」**。

原因是关键词匹配做不到语义判断，硬判会给出错误结论——把"没提到"当成"不合格"，会直接误导管理员。所以这个工具只做它擅长的事：找缺口。是否达标由模型结合档案说明。

返回结果里带了一句提示，明确告诉模型这一点：

> 以上只反映访客是否提到过，不代表是否达标——是否合适请结合宠物档案与访客具体情况判断。

工具层通过依赖注入拿到「执行 SQL」的能力，不关心底层是 OpenAPI 还是别的通道。这样单元测试可以直接替换成假实现，不需要连数据库。

4.3.2 Agent 循环

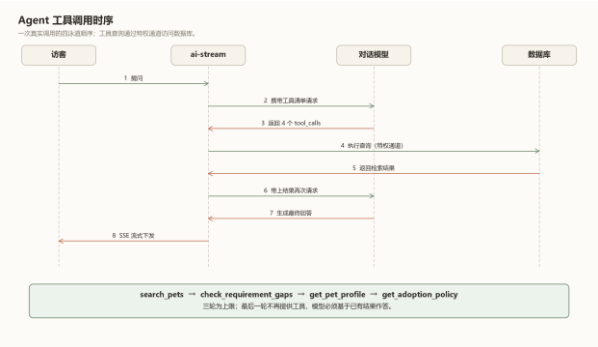


图 5　Agent 工具调用时序

循环本身不复杂：

```
messages = [系统提示, ...对话历史]
for round in 1..3:
    若 round < 3：带工具清单请求模型；最后一轮不带
    流式接收：正文 → 分片下发；tool_calls → 按 index 归并
    若本轮没有工具调用：结束
    执行工具 → 结果写回对话 → 进入下一轮
```

`MAX_TOOL_ROUNDS = 3`。**最后一轮不再提供工具**，强制模型基于已有信息作答。没有这一条，模型的自然倾向是"再查一次确认"，而每一轮都是一次完整的模型调用。

实测中模型会自主规划多步。以「我想领养一只猫，我租房住，房东同意，白天家里有人」为例，它依次调用了：

```
search_pets → check_requirement_gaps → get_pet_profile → get_adoption_policy
```

没有人告诉它这个顺序，也未固定流程分支。

4.3.3 RAG：切块与检索

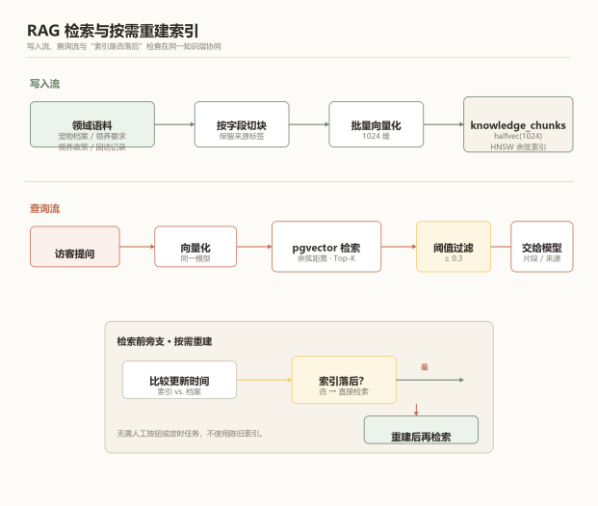


图 6　RAG 检索与按需重建索引

切块策略按字段拆分，而不是一只宠物一个大块。

宠物档案切成两块：一块是性格、健康、标签、年龄，另一块是领养要求。领养政策也按字段拆开（流程与要求、隐私说明、资料保存期限），回访记录每条单独成块。

这样做的原因：访客问「我适合养它吗」，需要命中的是**要求块**，而不是性格块。混在一起会让检索出来的片段又长又不聚焦，模型仍需自行筛选。当前语料切出 14 块（12 块宠物 + 2 块政策）。

每块都带**来源标签**（「喜豆的领养要求」「领养政策（隐私说明）」）。检索命中后，标签直接用作界面上的「依据」展示。

检索与阈值。 查询先向量化，再用余弦距离取最近邻，**相似度低于 0.3 的片段不返回**。优先返回空结果，而非以不相关内容填充。没有命中时，工具返回的提示里明确写着「不要凭印象作答」。

按需重建索引。 检索前先比较「索引最后更新时间」与「档案最后更新时间」，索引落后就重建。不依赖管理员点按钮，也不依赖定时任务——这两者都会出现"忘了执行"的窗口期，而按需重建不会有陈旧索引。

4.3.4 多模态的三处集成

语音输入。 用浏览器原生的 Web Speech API，不接第三方语音服务：零成本、无需额外密钥、音频不出浏览器。识别结果**只追加到输入框，不替用户发送**——语音识别必然出错，必须由用户核对后再提交。浏览器不支持时按钮直接隐藏，不显示一个点了没反应的按钮。

语音朗读。 每条回答下方一个「朗读这段回答」，同样用浏览器原生能力。

照片理解（管理员侧）。 管理员选好封面照片后点一下，视觉模型读图并写一段描述初稿填进描述框。提示词里明确禁止它臆测性格、病史、是否绝育、是否打过疫苗——这些照片看不出来。它只写毛色花纹、体型、耳尾特征、年龄感、精神状态。生成的是初稿，可随意修改。

照片在浏览器里先压到 768px、JPEG 0.82 再上传。原图动辄几 MB，base64 之后更大，请求体直接超限；视觉模型也不需要那么高的分辨率。

以图搜宠（访客侧）。 上传一张喜欢的猫狗照片，找出画面特征最接近的站内宠物。

这里有一个具体的技术障碍值得记录：宠物照片存在云存储里，公开地址是**带签名的临时链接**，服务端拼不出来（实测无签名访问 `tcb.qcloud.la` 返回 418、COS 直连返回 404）。

解法不是硬拼地址，而是：

- **缓存键用 storage_path**（稳定不变），不用会过期的签名地址；
- 前端每次把当前有效的签名地址一并传来，**只在缓存未命中时才使用**；
- 于是第一次查询逐张建索引会慢一些，之后全走缓存，

签名轮换也不会导致反复重算向量。

4.3.5 前端集成：SSE 事件协议

`ai-stream` 向前端下发的事件类型：

表 8　SSE 事件类型

事件	含义
`meta`	会话开始，报告是否演示模式、已加载工具数
`tool`	工具调用状态（running / done + 一行结果摘要）

事件	含义
`delta`	正文增量，前端累加
`sources`	本次回答引用的资料出处
done / error	结束

`sources` 由**服务端**在工具返回命中片段时下发，不依赖模型在正文里引用。模型可能忘了说，而「依据可查」是可解释性的硬要求，不能靠它自觉。

4.4 优化策略

4.4.1 成本控制

按需查询代替全量注入。 提示词里不再携带宠物档案，一次对话只在实际需要时查询相关数据。

只保存真实模型结果。 AI 摘要在改动后支持持久化，但只保存模型产出的结果。规则兜底的结果不保存——那种结果本来就便宜，存下来反而会让人误以为那是模型结论。

工具调用限三轮，避免反复查询。

访客侧限频：同一 IP 十分钟 15 次、每天 120 次。IP 只以 SHA-256 哈希存储。

嵌入计算的触发点最小化：向量只在语料变更时重算，不是每次查询都算。

4.4.2 延迟优化

流式输出而非等整段生成。 SSE 边生成边推，首字节远早于结束。

模型调用轮次与流式输出同时进行。 常见做法是「先非流式问一轮工具、再流式答一轮」，那样每次都多打一次模型、延迟翻倍。本项目采用流式聚合 `tool_calls` 分片的方式——多写一点代码，但正文能边生成边下发。

索引按需重建而非每次重建。 正常情况下检索直接走已有索引。

查询向量与文档向量分开缓存。 文档向量长期有效；签名地址的变化不影响缓存命中（缓存键是稳定的存储路径）。

实测数据：向量接口单次调用约 350ms；文本检索首次建索引 1145ms，命中缓存后明显更快。

4.4.3 输出质量

挡掉"调工具前的自言自语"。 模型在发起工具调用前常常先输出一句开场白，实测出现过英文的 `I'll look into suitable cats and the adoption requirements`。直接流出去会混进最终回答里。

处理方式是缓冲：带工具的轮次里正文先不立即下发，超过 200 字才认为"这是正文而不是开场白"并开始流式；较短的句子一旦本轮出现工具调用就整体丢弃。最后一轮（不带工具）直接流式。阈值定 200 字是因为实测的开场白都在 100 字以内，而真正的回答远超这个长度。最初定为 60 字，结果那句英文有 65 字，刚好越过阈值仍然漏了出去——这是对照真实输出调出来的参数，不是拍脑袋定的。

结构化输出约束。 摘要类任务要求模型输出固定结构的 JSON（结论、核对项、追问建议、风险点），并做字段级截断与清洗。模型失败时退回规则比对，并在界面上如实标注来源是"规则结果"而不是"AI 结论"。

明确的"不许编"约束。 系统提示里写了「凡涉及具体宠物、领养政策、回访情况，必须先查再答」「工具说没有，就如实说没有」。领养政策尚未填写时，实测模型回答的是「小院目前还没有公布领养流程、费用和回访安排的详细说明，我这边不能替它做推测」——而不是编一套收费标准出来。

4.4.4 稳定性：三层降级

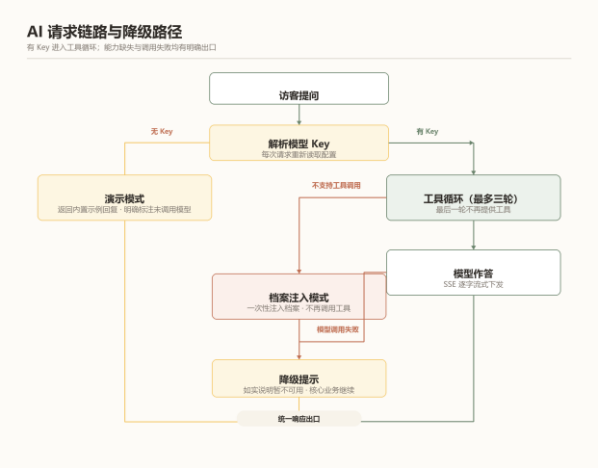


图 7　AI 请求链路与降级路径。三个降级分支分别在每次请求、模型调用失败、模型不支持工具调用时触发。目的是让 AI 匹配助手不会因为某一层故障而整体失效

表 9　三层降级行为

情况	行为
未配置模型 Key	进入演示模式，回复由内置示例生成，界面明确标注
模型调用失败	退回规则比对，并说明失败原因
模型不支持工具调用	退回"档案直接注入"模式，匹配助手仍可用

工具层自身也不抛异常：任何工具执行失败都转成`{ ok: false, message }`交给模型，让它可以换个方式继续，而不是让整轮对话崩掉。知识检索失败同样返回可读结果，由模型改用结构化查询。

核心业务永远不受影响： AI 服务全部不可用时，网站浏览、宠物档案、领养申请提交仍然正常。

4.4.5 隐私边界

访客侧对话只发送宠物公开档案与访客自述，不含任何申请人隐私数据。

审核助手会碰到其他申请人的数据。这里做了明确限制：**只把当前这位申请人的联系方式作为查询条件，不把其他人的姓名与联系方式写进模型上下文。**查「是否重复投递」时返回的是宠物名、提交时间、处理状态，不含联系方式。管理员本来就知道自己在审谁，多喂一份他人隐私没有必要。

访客侧刻意不提供「按联系方式查申请记录」这类工具。否则等于任何人都能查询"某人有没有来申请过"，属于隐私泄露。这类查询只保留在管理员侧。

以图搜宠的访客照片不写入云存储。 在浏览器里压缩后作为 data URL 发出，比对完即丢弃。

服务端会抓取前端提供的图片地址，因此加了两道限制：域名白名单（只允许本站云存储域名，且必须 https）、路径校验（地址路径必须包含对应的存储路径，且该路径必须真实存在于影像表中）。两道都通过才会发起请求。这部分有 15 项专门的测试覆盖绕过尝试，包括前缀伪装`evil-tcloudbasegateway.com`、后缀伪装`tcloudbasegateway.com.evil.com`、同域名但路径不含目标存储路径等。

五、核心功能实现

本章说明功能在界面上如何落地。业务部分从简，重点放在 AI 能力与界面的衔接上。

5.1 访客侧：从浏览到提交申请

5.1.1 页面结构

- `index.html`：首页介绍、数量概览与任务入口。
- `pets.html`：宠物筛选、档案详情与申请入口。
- `match.html`：AI 匹配对话与以图搜宠。
- `adoption.html`：领养流程、基本条件与申请修改入口。
- `stories.html`：已领养档案与回访记录。

- `about.html`：小院介绍、团队照片与联系方式。
- `admin.html`：独立管理页面，包含概览、档案、申请和设置。

旧首页锚点跳转到对应页面，保留查询参数；宠物详情支持 `?pet=编号`。本机预览不加载 CloudBase SDK，不连接业务或 AI 云端；编辑只存本浏览器，AI、图搜和语音识别禁用，避免审核时误提交或消耗模型额度。

5.1.2 浏览与筛选

宠物档案、影像、回访记录对匿名访客可读，权限由数据库的 RLS 策略放行。数据一次拉取后在浏览器内完成筛选与排序。

为什么不在数据库侧分页查询。 站点在册规模是数十只量级，一次拉取的体积远小于多次往返的开销。这个选择有明确前提：如果在册数量增长到数百只，需要改回服务端分页。

5.1.3 提交申请

申请不直接写数据库，而是经云函数 `yard-api` 提交。匿名角色对 `applications` 表的写权限已被回收，绕过前端也写不进去。

提交侧有多道约束：

表 10　提交侧的约束措施

措施	作用
蜜罐字段	正常用户不可见、不会填写；被填写则判定为机器人
最短填写时长	秒填秒交视为异常
浏览器指纹	与 IP 一起参与限频，避免单靠 IP 被绕过
多维限频	同一来源在时间窗口内的提交次数上限
服务端字段校验	长度、枚举值、联系方式归一化

提交成功后返回一枚**一次性编辑令牌**，用于申请人自助修改资料。数据库只存令牌的 SHA-256 哈希，有效期 24 小时。

5.1.4 自助修改

申请人凭令牌修改资料，可改字段是白名单：姓名、年龄、性别、联系方式、住房情况、养宠经验、家庭成员态度、现有宠物、补充说明。

明确不包含意向宠物、申请类型、内部状态、内部备注、以及任何令牌字段——这些改动应当由管理员经电话确认后进行。

实现上只提交**真正变动过**的字段，并且只覆盖申请人自己改过的部分，避免自助修改时把管理员已录入的内容冲掉。

5.2 管理员侧：后台与提醒

5.2.1 身份判定

管理员用「用户名 + 密码」登录，登录方式在 CloudBase 身份认证中开启。

身份判定分两层：前端据 `yard_administrators` 表决定是否进入后台，云函数内再用 `requireAdmin` 二次校验。**前端判定只是界面控制，真正的权限边界在服务端**，前端被篡改也拿不到管理能力。

这里有一处必须说明的实现约束：云函数内访问受限数据**必须走特权通道**，不能使用以匿名身份运行的数据库客户端。这个问题在 3.5 节已详述。

5.2.2 申请处理

收件箱按提交时间倒序展示。管理员可以更新处理状态、填写内部备注，每次变更写入 `application_events`，保留完整处理历史。

内部备注不会出现在企业微信提醒中，也不对访客可见。

5.2.3 宠物档案维护

支持新增与编辑档案、上传影像（含前端裁切）、维护回访记录。档案保存后，向量索引会在下次检索前自动重建，不需要管理员额外操作。

5.2.4 企业微信提醒

新申请提交后向小院的企业微信群推送提醒。提醒内容是**纯文本**，包含宠物、申请类型、时间与申请人填写的资料，供负责人联系审核；接收群须限制为有权处理申请的人员。内部备注、来源哈希与修改令牌不推送。

选择纯文本而非图文卡片是有原因的：企业微信群机器人对图文类消息有渲染限制，在普通微信客户端中会显示为「暂不支持此消息类型」，需要点进企业微信才能看。换成纯文本后，在微信侧可以直接读完。

发送结果（成功或失败原因）会回写到设置中，管理员在后台能看到最近一次的发送情况。**提醒失败不影响申请入库**，两者相互独立。

5.2.5 AI 配置入口

管理员在「网站设置」中填写两个 Key：对话模型与 TokenHub。两者独立保存，界面只显示掩码，不回显完整密钥。

密钥不进代码、不进仓库、不需要重新部署云函数，随时可更换。密钥存在数据库中一张受 RLS 保护的表里，匿名访客读不到。

旁边有一个「检查 AI 配置」按钮，实测两件事：

- 向量接口是否连通、输出维度是否符合预期
- **语义方向是否正确**——用三句固定文本（两句领养相关、一句无关），

校验相关相似度是否高于无关相似度

第二项是必要的：接口能返回向量，不等于检索方向正确。方向错了，检索会稳定地返回不相关内容，而且不会报任何错。

5.3 AI 功能的界面落地



图 8　访客侧 AI 匹配面板。访客描述自身情况后 AI 的完整回答，下方为朗读入口

5.3.1 匹配助手

访客在输入框描述自身情况，AI 流式作答。界面上有三处设计值得说明。

查询过程可见。 模型调用工具时，界面显示「正在查找宠物档案…」「正在按语义检索小院资料…」，查询完成后显示结果摘要（如「找到 3 只」）。AI 的等待过程因此不是黑箱——访客知道它在做什么，也能判断它查的是不是自己关心的东西。

依据可查。 回答下方列出本次引用的资料出处（如「喜豆的领养要求」「领养政策（隐私说明）」）。这些标签由服务端下发，**不依赖模型在正文里引用**——模型可能忘了说，而依据可查是可解释性的要求，不能靠它自觉。

演示模式如实标注。 未配置模型 Key 时，面板上明确显示「演示模式：回复由内置示例生成，未调用真实模型」。应让访客知悉这是示例，而非误认为是真实回答。

语音输入的识别结果只追加到输入框，**不替访客发送。**



图 9　管理员侧 AI 审核助手。逐条核对结果、「建议电话里问」、风险提示，以及「AI 审核过程」列出的本次查询项。摘要在生成后会保存，下次打开无需重新生成

5.3.2 审核助手

管理员在申请卡片上点「AI 摘要」，得到三块内容：逐条核对结果（满足 / 需确认 / 缺失 / 未提及，四色区分）、建议电话里问的问题、风险提示。下方另列「AI 审核过程」，说明本次实际查了哪几项。

摘要会持久化。 这是实际使用中发现的问题：初版算完就丢，每次打开申请都要重新生成，等十几秒，而且摘要本身是有价值的判断记录。现在生成后写回申请记录，下次打开直接显示，按钮文案变为「重新生成摘要」。

过期有提示。 如果申请资料在摘要生成之后被改动过（包括管理员代申请人更正），摘要上方会提示「旧结论的依据已经变了，建议重新生成」，但**不自动丢弃**旧结论——管理员可能正需要对照。

AI 不修改任何状态。 审核助手全程只读，处理状态始终由管理员手动更新。这条边界在系统提示与界面上都有体现。



图 10　以图搜宠结果。上传一张照片后，返回画面特征最接近的站内宠物与相似度。每只旁边提供「问问它」，可直接带着宠物名进入对话继续询问

5.3.3 以图搜宠

访客上传照片后，界面显示比对进度与匹配结果（缩略图、宠物名、画面相似度）。命中的宠物可一键「问问它」，把宠物名带进对话继续问匹配情况。

照片在浏览器里压缩后作为 data URL 发送，**不写入云存储**。第一次查询需要给站内宠物照片建立索引，会明显慢一些，界面有相应提示。



图 11　语音输入与朗读。输入框旁的语音按钮（录音中呈高亮呼吸态），以及回答下方的朗读入口

5.3.4 语音输入与朗读

语音按钮在浏览器不支持时直接隐藏，不显示一个点了没反应的按钮。麦克风权限被拒时明确提示怎么开启。

识别结果追加到输入框后可以继续编辑。**不做"说完自动发送"**——语音识别必然有错字，自动发送会把错误直接提交出去。

每条回答下方有「朗读这段回答」，使用浏览器原生语音合成，不依赖外部服务。

5.3.5 照片理解（管理员侧）

管理员在编辑宠物档案时选好封面照片，点「用封面照片自动写一段」，视觉模型写出的描述会填入描述框，管理员可以随意修改。

界面上明确写着「只描述照片中可见的内容」——性格与病史无法从照片判断，仍需人工填写。这条提示既是对管理员的说明，也是对模型输出的预期管理。

六、测试与部署

6.1 测试策略

不使用测试框架。 所有验证脚本都是普通 Node 脚本，直接 `node tools/xxx.cjs` 运行，不引入 Jest、Mocha 之类的依赖。理由很直接：小院没有专职开发者，测试框架的配置、版本升级、断言语义都是额外负担；而这些脚本只要能执行、能报告通过项数，作用并无差别。

失败信息必须能定位问题。 断言失败时写明期望值与实际值，而不是只回一句"失败"。例如图片地址校验的测试会明确指出是前缀伪装还是后缀伪装没有被拦住。

按风险分三档：`npm test` 完全离线；`npm run test:online` 只读线上；`npm run test:live` 会向线上写入数据，只由委托方在部署后执行。本轮仅运行第一档，覆盖如下：

表 11　测试覆盖

类型	脚本	项数	验证内容
结构	`check-admin-settings.cjs`	16	后台表单结构与控件完整性
结构	`check-multipage.cjs`	277	七页身份、导航、共享资源与功能区隔离
行为	`test-local-review.cjs`	17	本机禁云、缺少宿主安全与预览路径白名单
合规	`check-repo-compliance.cjs`	31	密钥、README、Git 记录、仓库整洁
单元	`test-knowledge.cjs`	23	切块、索引重建、阈值过滤、降级
单元	`test-vision-guards.cjs`	15	图片地址白名单与各类绕过尝试
单元	`test-tc3-signing.cjs`	51	云 API 签名
单元	`test-wecom-notify.cjs`	56	提醒内容与降级
单元	`test-admin-applicant-edit.cjs`	24	代改资料的字段白名单

合计 **510 项通过、0 失败**，一次约 7–8 秒；另有一条团队信息待补提醒。只读线上档保留 `check-release-sync.cjs` 完整版与 `check-ai-preview.cjs`。云端写入档保留 `probe-browser-path.cjs`、`test-ai-chat.cjs`、`test-ai-tools.cjs` 与 `verify-anon-lockdown.cjs`：会提交测试申请、调用真实模型并写限频行，或尝试数据库写入。本轮两档均未执行，不将历史线上结果计入上述合计。发布源本地比对另保留 `--offline` 命令。

单元测试用依赖注入，不用打桩库。 工具层、知识层在构造时接收「执行 SQL」与「向量化」两个函数，测试时传入假实现即可，既不需要连数据库也不需要网络，因此可以反复跑、不受接口限频影响。

端到端测试打的是线上环境，不是本地模拟。只有真实调用才能验证匿名登录、云函数鉴权、网关 CORS、SSE 流式这些环节。

6.2 测试拦住的问题

下表列出测试实际拦下的缺陷及其对应的验证方式：

表 12　测试拦住的缺陷

缺陷	性质	被哪类验证拦住
云函数里用了匿名身份的数据库客户端	后台多个功能长期不可用	端到端（真实调用才暴露）
数据库列名写错（content 应为 body）	AI 查询静默失败	工具冒烟测试
引用了不存在的列（updated_at）	索引重建失败	工具冒烟测试
漏写 res.end()	响应不关闭，访客每次等满超时	端到端（表现为测试超时）
前端改了但没同步发布源	线上仍是旧版本	同步检查
后台表单字段被误删	输入框静默消失	结构检查
图片地址白名单域名写错	以图搜宠全部失败	端到端
模型开场白混入回答	回答里出现英文句子	端到端（人工看输出）

其中**工具冒烟测试**（`/health/tools`）针对「手写 SQL 易写错列名」设计：逐个执行工具，列名错误立即暴露，而非等到对话阶段以「无法回答」的形式出现——后者难以定位到具体 SQL。加入后一次运行即发现两个列名错误，其中一个所在工具此前从未被调用，因而长期未暴露。

6.3 部署

6.3.1 部署顺序

依次为：数据库迁移 → 部署 `yard-api`（Event）→ 部署 `ai-stream`（HTTP）→绑定 HTTP 访问服务 → 发布前端。具体命令见仓库 `package.json` 的脚本定义。

6.3.2 部署中遇到的两个问题

第 3、4 步必须分开执行。 若在第 3 步的部署命令中直接带 `--path`，平台会建立一条触发类型错误的绑定，访问时返回 `FUNCTIONS_PARAM_INVALID: FunctionType parameter is invalid`。正确做法是先单独部署函数，再用 `service create` 绑定访问服务。

HTTP 云函数的两个必需配置。 一是必须有 `scf_bootstrap` 启动文件（已随代码提交），应用监听 9000 端口；二是默认超时仅 3 秒，SSE 会被中途截断，需在配置中改为 60 秒。两项缺任一，本地验证通过而线上不可用。

6.3.3 发布源与同步检查

前端发布的是 `release/` 目录，根目录为源码。委托方审核后统一同步七个 HTML 页面、`app.js`、`site-config.js`、`styles.css` 与 `assets/`，再执行部署和线上哈希验收。本轮只修改并提交本地源码，不改动发布源，也未发布；新页面的线上地址待部署后生效。

这里有两个容易漏的环节，都不会报错，只表现为"我明明改了，线上怎么没变"：

1. 改了根目录文件，忘了同步到 `release/`
2. 同步了 `release/`，忘了执行部署

因此专门写了同步检查：既比对根目录与 `release/`，也抓取线上文件比对哈希，**两个环节分别报错**，直接写明是漏了哪一步。

6.4 环境复现

仓库根目录提供 `.env.example`，列出全部环境变量名、用途与填写位置，不含任何真实值；真实的 `.env` 已被 `.gitignore` 排除。依赖清单在 `package.json`，两个云函数各自也有 `package.json`。

有两项配置**只在运行时才暴露问题**，部署成功不代表能用：

匿名登录未开启时，`signInAnonymously()` 会直接报「登录方式未开启」。**云函数安全规则**的默认值把匿名用户排除在外，表现为访客点提交申请没有反应。控制台入口在「云函数 → 鉴权设置」，位置较深，因此提供了命令行工具：

```
node tools/fn-security-rule.cjs describe   # 查看当前规则
node tools/fn-security-rule.cjs apply      # 设为全放行
```

安全规则设为全放行是安全的：云函数内部自己做鉴权，管理动作走 `requireAdmin`，公开动作有限频与输入校验。这正是官方文档推荐的「更精细的权限控制在云函数内部实现」。

而 `probe-browser-path.cjs` 会完整走一遍**匿名登录 → 调云函数 → 读数据 → 提交申请**，全部通过才视为可用。

6.5 上线后的实际使用

平台已在为小院实际服务：宠物档案在真实维护，领养申请在真实提交，新申请会推送到小院的企业微信群。

AI 能力的部分改进直接来自实际使用——例如审核摘要的持久化，就是因为管理员反映"每次打开都要重新生成一次"才加上的。这类问题在测试里不会暴露，只有真正用起来才会遇到。

七、创新点

赛事要求明确列出技术创新点与应用创新点，这里分别说明。

7.1 技术创新点

一、将 AI 的数据访问实现为工具，而非扩大提示词。

常见做法是把领域数据拼进提示词，规模增大即触及上下文上限。本项目改为由模型自主调用查询工具，AI 能力与数据规模由此解耦：在册宠物从 6 只增至 600 只，提示词无需改动。

二、划定规则与模型的分工边界。

`check_requirement_gaps` 只判断访客是否提及某件事，不判断是否达标，并将这一点写入返回给模型的提示中。关键词匹配不具备语义判断能力，越界即会把「未提及」当作「不合格」，直接误导管理员。**明确工具不应做什么，与明确它应当做什么同等重要。**

三、向量索引按需重建，不依赖人工操作或定时任务。

检索前比较索引与语料的更新时间，落后即重建。人工点按与定时任务都存在遗漏窗口，按需重建不存在该窗口，也不会使用陈旧索引。向量缓存的键采用稳定的存储路径而非签名地址，签名轮换不影响缓存命中。

四、两层地址校验，防止服务端被用于抓取任意网址。

以图搜宠需要服务端抓取前端提供的图片地址，属于典型的 SSRF 面。限制分两道：域名白名单（仅允许本站云存储域名且必须为 https）、路径校验（地址路径须包含目标存储路径，且该路径须真实存在于影像表）。两道均通过才发起请求，并有 15 项测试覆盖各类绕过尝试。

五、全链路降级与「不编造」约束。

模型、工具、向量服务任一不可用时，均有对应的降级行为，不影响核心业务。提示词层面要求信息缺失时如实说明：实测在领养政策未填写时，模型回答的是「小院尚未公布，建议直接联系确认」，未编造收费与回访安排。

7.2 应用创新点

一、AI 落在「匹配」环节，而非通用问答。

宠物领养平台接入聊天机器人并不困难，但难以解决问题。本项目的 AI 围绕一个目标：判断什么样的领养更可能成功。访客不知如何选择、管理员逐条核对耗时这两个痛点，都源于这一判断的缺失。

二、访客侧与管理员侧共用同一工具层。

两侧面向不同使用者，底层是同一批只读工具。访客侧由模型自主规划多步查询，管理员侧由模型自主规划审核步骤，复用的是同一套数据访问能力，差异仅在提示词与输出结构。

三、将难以表述的需求转为可检索输入。

访客通常能说出感受而说不出条件：「想找一只安静点的」，档案中记录的是「不爱动，大部分时间在睡觉」。语义检索衔接了这类描述性表达；以图搜宠进一步降低了表达门槛，无法表述时可提交一张照片。

四、AI 的结论可核查。

回答下方列出引用的资料出处，审核摘要列出本次查询项。除透明性外，它使使用者能够判断结论不准的原因：是 AI 未查询，还是资料本身缺失。两者的处理方式不同。

五、隐私边界落在功能设计上。

审核助手需查询是否重复投递，但返回内容为宠物名、时间、状态，**不含其他申请人的姓名与联系方式**；访客侧不提供同类工具，避免任何人都能查询他人是否申请过。约束由接口设计保证，不依赖使用者的自觉。

八、总结与展望

8.1 做了什么

参赛期间的工作分两块。

AI 能力：六项功能覆盖访客侧与管理员侧——领养匹配助手、申请审核助手、RAG 语义检索、语音输入与朗读、照片理解、以图搜宠。底层是工具层、知识层、Agent 层、多模态层四层结构。

工程质量：引入 ESLint 与 Prettier，补齐环境变量模板；本轮离线回归 510 项通过，修复若干历史缺陷，其中一处曾导致后台多个功能长期不可用。

8.2 反思

原生 JavaScript 是有代价的选择。 它让首屏最轻、代码最易维护，但也意味着前端工程规范那部分拿不到分。我们仍然认为这个交换划算，但这一点应当说清楚，而不是含糊带过。

RAG 的效果取决于语料。 语义检索能把"安静点的"匹配到"不爱动"，但如果档案里根本没写性格，再好的检索也无能为力。领养政策尚未填写完整，AI 问到流程与费用时只能回答"还没公布"。**这是数据问题，不是技术问题**，也是下一步最该补的。

AI 不能替代判断。 审核助手能整理信息、指出缺口、建议追问什么，但它不知道申请人电话里的语气，也不会为一个决定承担后果。所以它只做整理与提示，决定权留在管理员手里。

8.3 展望

工具层可以对外暴露。 当前工具通过 Function Calling 在云函数内部调用。接口设计与 MCP 是同一思路（名称、描述、JSON Schema 参数），将来若要接入其他 AI 客户端，改造成 MCP Server 的成本很低。

管理员权限需要细分。 目前管理员之间权限相同，随着小院人员增加，需要区分"能看"和"能改"。数据库已经启用了 RLS，加一层角色判定即可，不需要改动架构。

语料需要持续补充。 领养政策、回访记录、常见问题问答——这些是 RAG 效果的真正来源，也是当前最明显的短板。

可以往前再走一步。 目前 AI 服务于"申请前"和"审核中"，领养之后的跟踪回访仍是人工。这里的空间不在于做更多功能，而在于把已有数据用起来——例如根据回访记录提示"这只动物已经一年没有回访"。